

Testing the lists

The following set of functions demonstrates the various checks on lists:

- *atom(v)* - returns 'true' if 'v' is an atomic element; 'false' otherwise
- *listp(L)* - returns 'true' if 'L' is a list; 'false' otherwise
- *member(v, L)* - returns 'true' if 'v' is a member of list L; 'false' otherwise
- *some(p, L)* - returns 'true' if predicate 'p' is true for at least one member of list L; 'false' otherwise
- *every(p, L)* - returns 'true' if predicate 'p' is true for all members of list L; 'false' otherwise

```
$ maxima -q
(%i1) atom(5);
(%o1) true
(%i2) atom([5]);
(%o2) false
(%i3) listp(x);
(%o3) false
(%i4) listp([x]);
(%o4) true
(%i5) listp([x, 5]);
(%o5) true
(%i6) member(x, [a, b, c]);
(%o6) false
(%i7) member(x, [a, x, c]);
(%o7) true
(%i8) some(primep, [1, 4, 9]);
(%o8) false
(%i9) some(primep, [1, 2, 4, 9]);
(%o9) true
(%i10) every(integerp, [1, 2, 4, 9]);
(%o10) true
(%i11) every(integerp, [1, 2, 4, x]);
(%o11) false
(%i12) quit();
```

List recreations

Next is a set of functions operating on list(s) to create and return new lists:

- *cons(v, L)* - returns a list with 'v', followed by members of L
- *endcons(v, L)* - returns a list with members of L followed by 'v'
- *rest(L, n)* - returns a list with members of L, except the first 'n' members (if 'n' is non-negative), otherwise except the last '-n' members. 'n' is optional, in which case, it is taken as 1
- *join(L1, L2)* - returns a list with members of L1 and L2 interspersed
- *delete(v, L, n)* - returns a list like L but with the first 'n' occurrences of 'v' deleted from it. 'n' is optional, in which case all occurrences of 'v' are deleted
- *append(L1, ..., Ln)* - returns a list with members of L1, ..., Ln, one after the other
- *unique(L)* - returns a list obtained by removing the duplicate

members in the list L

- *reverse(L)* - returns a list with members of the list L in reverse order

```
$ maxima -q
(%i1) L: makelist(i, i, 1, 10);
(%o1) [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
(%i2) cons(0, L);
(%o2) [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
(%i3) endcons(11, L);
(%o3) [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11]
(%i4) rest(L);
(%o4) [2, 3, 4, 5, 6, 7, 8, 9, 10]
(%i5) rest(L, 3);
(%o5) [4, 5, 6, 7, 8, 9, 10]
(%i6) rest(L, -3);
(%o6) [1, 2, 3, 4, 5, 6, 7]
(%i7) join(L, [a, b, c, d]);
(%o7) [1, a, 2, b, 3, c, 4, d]
(%i8) delete(6, L);
(%o8) [1, 2, 3, 4, 5, 7, 8, 9, 10]
(%i9) delete(4, delete(6, L));
(%o9) [1, 2, 3, 5, 7, 8, 9, 10]
(%i10) delete(4, delete(6, join(L, L)));
(%o10) [1, 1, 2, 2, 3, 3, 5, 5, 7, 7, 8, 8, 9, 9, 10, 10]
(%i11) L1: rest(L, 7);
(%o11) [8, 9, 10]
(%i12) L2: rest(rest(L, -3), 3);
(%o12) [4, 5, 6, 7]
(%i13) L3: rest(L, -7);
(%o13) [1, 2, 3]
(%i14) append(L1, L2, L3);
(%o14) [8, 9, 10, 4, 5, 6, 7, 1, 2, 3]
(%i15) reverse(L);
(%o15) [10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
(%i16) join(reverse(L), L);
(%o16) [10, 1, 9, 2, 8, 3, 7, 4, 6, 5, 5, 6, 4, 7, 3, 8, 2, 9, 1, 10]
(%i17) unique(join(reverse(L), L));
(%o17) [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
(%i18) L;
(%o18) [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
(%i19) quit();
```

Note that the list L is still not modified. For that matter, even L1, L2, L3 are not modified. In fact, that is what is meant when we state that all these functions recreate new modified lists, rather than modify the existing ones.

List extractions

Here is a set of functions extracting the various members of a list. *first(L)*, *second(L)*, *third(L)*, *fourth(L)*, *fifth(L)*, *sixth(L)*, *seventh(L)*, *eighth(L)*, *ninth(L)*, and *tenth(L)*, respectively return the first, second, ... member of the list L. *last(L)* returns the last